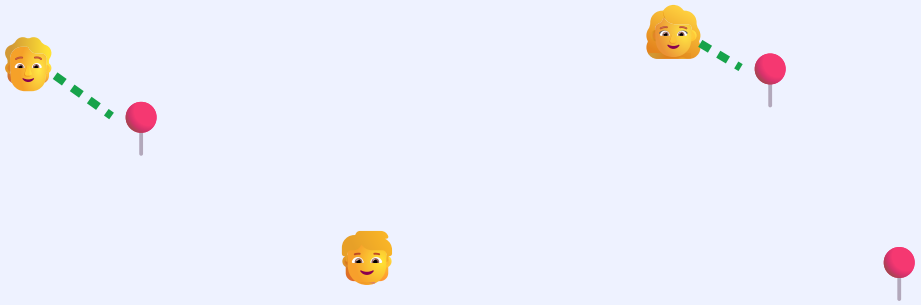
비슷한 점끼리
같이 걷게 하는 학습 규칙

25-fall-recommender · Cluster-Coupled Adam(CCA)을 그림으로

01 · 먼저 알아 둘 것

추천 모델 = 지도 위의 점들

사람과 장소가 모두 지도 위의 점이에요. 가까우면 "좋아할 것 같다".



점의 좌표 = 임베딩 · 사람 29,858명, 장소 40,981곳 (Gowalla)

02 · 학습이란

점이 조금씩 걸어가는 것

틀릴 때마다 점을 살짝 옮겨요. 한 걸음을 정하는 규칙이 **옵티마이저**.



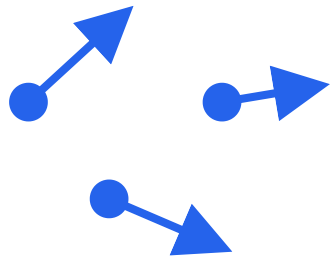
한 걸음, 한 걸음... (Adam이 가장 흔한 규칙)

03 · 핵심 아이디어

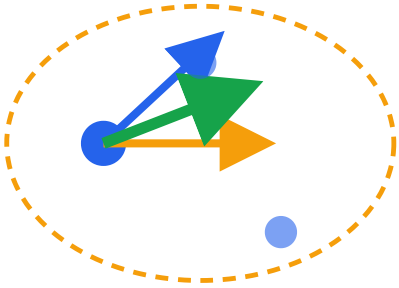
혼자 걷지 말고, 무리의 방향도 듣기

비슷한 점끼리 무리를 짓고, 내 화살표에 무리 평균 화살표를 조금 섞어요.

기존 Adam: 각자



CCA: 나 + 무리

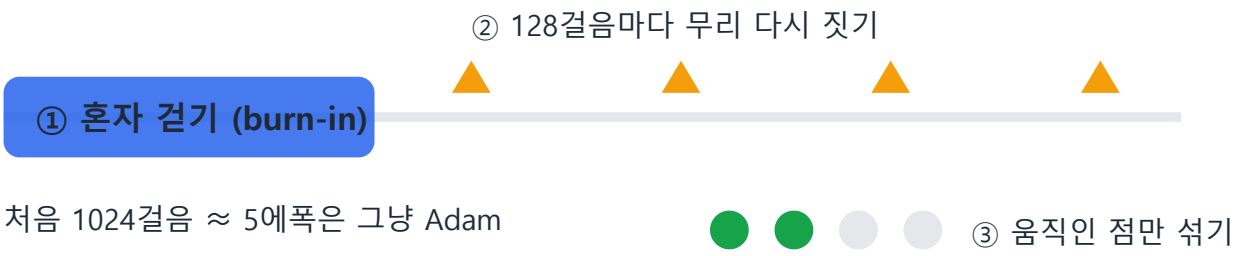


내 방향 × (1−α) + 무리 평균 × α = 실제 걸음

04 · 언제, 누구와

세 가지 안전장치

처음엔 혼자 걷고, 무리는 가끔 다시 짓고, 이번에 안 움직인 점은 건드리지 않아요.



무리 짓는 기준은 "최근 걸어온 방향의 평균"(EMA gradient), 무리 수는 처음 한 번 Elbow로 정해요.

05 · 부품 지도

파일마다 맡은 역할

main.py
지휘자. 에폭마다 학습 → 채점 → 기록

parse.py · world.py
설정판. 옵티마이저, α, 무리 수, 걸음 간격

dataloader.py
지도 펴기. 누가 어디를 갔는지 그래프로

model.py
LightGCN. 이웃 좌표를 평균해 점의 위치 계산

optimizers.py
★ 걸음 규칙. 무리 짓기 + 섞기 + Adam

Procedure.py · utils.py
채점표. Precision·Recall·NDCG, CSV 성적표

06 · 그래서 됐어?

결승선은 Adam과 비슷

끝 점수는 Adam과 비슷했어요. 발표 그래프의 "초반에 더 빠름"은 다시 돌려 보니 재현되지 않았어요.



⚠️ 다시 돌려 보니(노트북 설정, 10에폭) 처음 약 5에폭은 CCA도 그냥 Adam(①)이라 두 곡선이 겹쳤고, 그 뒤엔 CCA가 살짝 낮았어요(Recall 0.1203 vs 0.1218).

07 · 직접 해 보기

이 순서로 열어 보세요

1) `README.md` → 2) `code/optimizers.py`의 `ClusterCoupledAdam.step` → 3) `code/utils.py`의 `BPRLoss`

```
cd code        # ← 꼭 code 폴더 안에서! (데이터 경로가 ../data)
python main.py --dataset gowalla --model lgn --recdim 32 --layer 1 \
  --epochs 10 --bpr_batch 4096 --testbatch 512 \
  --optimizer cluster --alpha 0.25 --num_clusters 64 \
  --recluster_interval 128 --cluster_warmup 1024
# 비교: --optimizer adam   ·   결과: code/runs/*.csv
```

✅ HR@20(추천 20개 안에 정답이 하나라도 든 사람의 비율)은 계산 오류를 고쳐 다시 돌렸어요: Adam 0.463, CCA 0.458 (10에폭).

ELI5 · 큰 그림, 적은 글 · 출처: 팀 저장소 코드와 발표 자료(5조) · 내 역할: 실험·하이퍼파라미터 분석·수식 검증